

原論文・添付原実装に基づく是正仕様

FilterReg と Analytic-CPD

2 次元・3 次元の剛体位置合わせと
解析非剛体位置合わせ

C++ / nanobind および Rust / PyO3
独立数値核・共通 Python 操作・pixi 環境

文書の位置付け	目的・要求・要件、定式化、実装選択、原コードとの差分、再実行可能な検証、完了条件の判定を一体で記述します。
対象版	acpd-filterreg 0.2.0。以前の hybridreg および acpd_filterreg_suite とは区別します。
資料の基準	ユーザー添付の論文 2 件と原コード 4 件です。遠隔リポジトリの最新版に置き換えていません。
総合状態	DoD 全項目の完了には未到達です。 実 nanobind 接続のビルドと pixi 依存解決が未達です。Rust のコンパイル必須のみ、ユーザーの先行指示により免除されています。

重要な区別

数式に対応するコードがあること、原コードの計算結果に近いこと、Python 拡張をビルドできること、対象データで精度が改善することは別々の命題です。本書はこれらを同一視しません。特に、格子フィルタを全対評価や半径探索で代用した前回版は、今回の忠実性の基準を満たしていませんでした。

目次

1	目的、要求、要件と受入範囲	2
2	記号、変換方向、座標系	2
3	CPD 方向の確率モデルと E-step	3
4	FilterReg 方向の確率モデルとモーメント	4
5	事後重心への集約と分散更新の導出	5
6	ガウス和をフィルタに変換する入力拡張	6
7	Permutohedral 格子：埋込みと単体補間	6
8	Splat・Blur・Slice と原コードの最適化	7
9	剛体 M-step：twist と局所更新	8
10	点対線・点対面と任意の閉形式解	9
11	Analytic-CPD の構造化 Taylor 写像	10
12	無正則化の重み付き解析 M-step	10
13	次数継続：原コードの剰余割当まで定義	11
14	逐次合成、最良状態、停止判定	12
15	剛体初期化を固定した非剛体モード	13
16	再適用、微分、保存された写像の意味	14
17	入力契約、失敗時の扱いと数値限界	15
18	計算量と実装上の加速	16
19	構成、責務分離、両言語の対応	17
20	操作仕様と環境の再現	18
21	計算量と性能の主張の範囲	19
22	原資料との差分と選択の根拠	20
23	厳格レビューと是正内容	20
24	原実装比較と実行済み試験	21
25	合成データでの精度監査：悪化例も保持	22
26	完了条件の判定と残る阻害	23
27	資料識別、ライセンス、再照合の手順	25
	参考資料	26

1 目的、要求、要件と受入範囲

1.1 ユーザーの目的

目的は、点群間の未知対応を確率的に扱いながら、まず大域的な回転・並進を推定し、その後に滑らかな残差変形を解析写像で推定することです。元の `cpd3d_rs` のように Python から利用でき、同じ操作を 2 次元・3 次元、C++・Rust の両方で使えるようにします。[8]

今回の是正では「動けばよい別方式」ではなく、FilterReg のフィルタ型 E-step と twist 型 M-step、Analytic-CPD の CPD 事後・Taylor 基底・無正則化重み付き当てはめ・逐次合成を識別可能な形で実装することを要求します。これらはそれぞれの論文の主要構成です。[1, 2]

1.2 機能上の要件

モード	定義
<code>rigid</code>	FilterReg で $R \in \text{SO}(d), t \in \mathbb{R}^d$ を推定します。拡大縮小は推定しません。
<code>analytic</code>	与えられた初期姿勢に Analytic-CPD の解析更新を合成します。FilterReg は実行しません。
<code>nonrigid</code>	FilterReg を一度実行し、その最終姿勢を固定して Analytic-CPD を開始します。解析段階から剛体段階へ戻りません。

各モードは $d = 2, 3$ を扱います。固定点と移動点の点数は異なってよく、同じ点数でも行番号による対応は仮定しません。剛体段階は点对点と、2D の点对線・3D の点对面を扱います。非剛体段階は論文の二乗ユークリッド距離による解析当てはめです。

非機能要件は、数値核と言語接続の分離、入力・出力の所有権の明確化、設定の一貫性、再適用可能な変換の保存、例外処理、原資料から試験までの追跡性です。Rust から C++ へ処理を委譲すること、Python の代替計算へ黙って切り替えることは禁止します。

1.3 忠実性の意味と対象外

ここでの忠実性は、対象の数式・アルゴリズムに対応する処理と、その対応を確認する証拠を備えることです。原コードにある添字不整合や次元の固定値まで模倣することではありません。差分は第22節で明記します。

原論文の GPU、関節ロボット、変形グラフ、学習特徴による大域的初期化、全実データ実験および公表速度の再現は対象外です。2D への拡張は同じ確率式と $\text{SE}(2)$ の運動学から導出します。FilterReg 原論文の全機能を再現したという意味ではありません。

2 記号、変換方向、座標系

固定点群を $X = \{x_n\}_{n=1}^N \subset \mathbb{R}^d$ 、移動点群を $Y = \{y_m\}_{m=1}^M \subset \mathbb{R}^d$ とします。変換方向は常に $Y \rightarrow X$ です。本書は Analytic-CPD の記号に統一しています。FilterReg 原論文では X が移動側、 Y が観測側であり、記号だけを転記すると方向が逆になる点に注意が必要です。[1, 2]

記号	意味
p_m	現在の移動点。初期移動点 y_m と区別します。
K_{mn}	$\exp[-\ p_m - x_n\ ^2 / (2\sigma^2)]$ 。正規化係数を除いたガウス親和度です。
P_{mn}, Q_{mn}	CPD 方向、FilterReg 方向の事後責任度です。同じものではありません。
ρ_m, b_m, h_m	責任度の行和、固定座標の 1 次統計、固定座標の二乗ノルム統計です。
$z_m = b_m / \rho_m$	$\rho_m > 0$ のときの事後重心です。
$q, S_{d,q}$	Taylor 最大総次数、総次数 q 以下の基底数です。
$\mathcal{A}_k, C_k$	反復 k の解析写像、保存用の恒等写像との差の係数です。
$T_F, \mathcal{N}$	固定した剛体変換、共通の座標正規化です。

数式の点は列ベクトルです。Python 配列は 1 点を 1 行として保持し、剛体変換を `points @ rotation.T + translation` で評価します。回転行列を転置した値として保存することはありません。

2.1 共通正規化の明示的な定義

論文は共通の正規化座標と原点中心の Taylor 展開を要求しますが、尺度の一意的な計算式までは固定していません。本実装では以下を統合仕様として定義します。[2]

$$c_X = \frac{1}{N} \sum_n x_n, \quad s = \sqrt{\frac{1}{N} \sum_n \|x_n - c_X\|^2}, \quad \mathcal{N}(p) = \frac{p - c_X}{s}. \quad (2.1)$$

両点群に同一の c_X, s を適用します。別々に中心を引いて相対並進を消すことはありません。 s が非有限、ゼロまたは数値的に表現不能な場合は例外とします。正規化後の展開中心は $c = 0$ です。 c_X は世界座標の中心、 c は Taylor 展開中心であり、区別します。

明示指定の `sigma2` は入力座標の二乗単位です。数値核では σ^2/s^2 へ変換します。一方 `min_sigma2` は正規化座標の下限です。返却する分散と履歴の RMS は元の単位に戻します。負の対数尤度は正規化座標における診断値であり、異なる正規化を使う実行間の尺度不変な比較量ではありません。

3 CPD 方向の確率モデルと E-step

現在の移動点を混合分布の中心とし、固定点を観測として扱います。論文の外れ値モデルを用いると、正規化座標における評価式は

$$p(x_n) = \frac{w}{N} + \frac{1-w}{M} (2\pi\sigma^2)^{-d/2} \sum_{m=1}^M K_{mn}, \quad 0 \leq w < 1 \quad (3.1)$$

です。ここから Bayes 則により

$$P_{mn} = \frac{K_{mn}}{\sum_{j=1}^M K_{jn} + c_C}, \quad (3.2)$$

$$c_C = (2\pi\sigma^2)^{d/2} \frac{w}{1-w} \frac{M}{N} \quad (3.3)$$

を得ます。 $w = 0$ では各列の和が 1、 $w > 0$ では 1 以下です。これが Analytic-CPD で保持すべき CPD の事後層です。[2, 式 (2)–(5)]

必要な統計量は

$$\rho_m = \sum_n P_{mn}, \quad b_m = \sum_n P_{mn} x_n, \quad h_m = \sum_n P_{mn} \|x_n\|^2, \quad N_P = \sum_m \rho_m \quad (3.4)$$

です。 b_m を行に並べた行列が PX です。 h_m を保持すれば、分散更新のために列和を別途返却しなくても $\sum_m h_m = \sum_n (P^T \mathbf{1})_n \|x_n\|^2$ を得られます。

3.1 直接計算と数値安定化

ACPD の既定・唯一の事後計算は全点对を評価する直接法です。大きな $M \times N$ 行列を保存せず、観測点ごとに責任度を正規化して統計量を加算します。論文付録 A・B.4 が述べる逐次集計と同じ式です。格子近似や半径打切りは ACPD へ導入していません。[2]

分母の計算では $\ell_m = -\|p_m - x_n\|^2 / (2\sigma^2)$ と $\ell_c = \log c_C$ の最大値 a を引き、

$$P_{mn} = \frac{\exp(\ell_m - a)}{\sum_j \exp(\ell_j - a) + \exp(\ell_c - a)} \quad (3.5)$$

と評価します。 $w = 0$ は $\ell_c = -\infty$ として扱います。これは定式化の変更ではなく、同じ確率の数値評価です。距離自体のオーバーフローや、表現可能な支持がない場合には例外とします。

3.2 外れ値定数の解釈

論文で使われる w/N は指定した座標系に依存する背景項です。 $\mathbb{R}^d$ 全体に積分して 1 になる一様分布ではありません。厳密な生成密度として解釈するには有限観測領域とその体積を別に定義する必要があります。本版は論文の登録用規約を維持し、背景体積を新しい推定変数として加えていません。したがって、 w を物理的な外れ値率の校正済み推定値とみなしたり、正規化を変えて同じ尤度尺度と解釈したりしません。

4 FilterReg 方向の確率モデルとモーメント

FilterReg では固定点群が混合分布の中心で、移動点をその分布の高い位置へ移動させます。CPD とは条件付けの方向が逆です。[1, §3.1–3.2]

$$p(p_m | X) = \frac{w}{M} + \frac{1-w}{N} (2\pi\sigma^2)^{-d/2} \sum_n K_{mn}, \quad (4.1)$$

$$Q_{mn} = \frac{K_{mn}}{\sum_j K_{mj} + c_F}, \quad c_F = (2\pi\sigma^2)^{d/2} \frac{w}{1-w} \frac{N}{M}. \quad (4.2)$$

正規化係数を除いたガウス和を

$$m_{0m} = \sum_n K_{mn}, \quad m_{1m} = \sum_n K_{mn} x_n, \quad m_{2m} = \sum_n K_{mn} \|x_n\|^2 \quad (4.3)$$

とすると、

$$\rho_m = \frac{m_{0m}}{m_{0m} + c_F}, \quad z_m = \frac{m_{1m}}{m_{0m}}, \quad b_m = \frac{m_{1m}}{m_{0m} + c_F}, \quad h_m = \frac{m_{2m}}{m_{0m} + c_F} \quad (4.4)$$

です。 $b_m = \rho_m z_m$ なので、両手法の M-step は同じ統計の型を使えます。しかし E-step の正規化は共通化しません。CPD で FilterReg の行正規化を使うと別のモデルになります。

原論文の式 (6) では正規化済み Gaussian PDF を使い、そのとき $c = \frac{w}{1-w} \frac{N}{M}$ です。本書ではフィルタに渡す値が非正規化の K なので、 $(2\pi\sigma^2)^{d/2}$ を外れ値定数側に移しています。[1]

4.1 法線統計と支持がない点

固定点の単位法線を n_n とし、

$$m_{nm} = \sum_n K_{mn} n_n, \quad \bar{n}_m = m_{nm} / m_{0m} \quad (4.5)$$

を計算します。論文式 (9) 通りの重み付き平均です。平均後に勝手に単位長へ再正規化しません。向きの異なる法線が相殺して $\|\bar{n}_m\|$ が小さい場合、その点の平面拘束も弱くなります。

格子近似では、クエリ点に観測側の支持が存在せず $m_{0m} = 0$ となることがあります。この行の責任度を 0 とし、直接法へ切り替えません。 $w = 0$ の場合、その点の対数尤度は $-\infty$ であり、返却する診断値では非有限の負の対数尤度を None にします。有効な拘束が不足すれば剛体更新は例外になります。

5 事後重心への集約と分散更新の導出

以下は CPD の P 、FilterReg の Q のいずれにも適用できる代数です。責任度を一般に $W_{mn} \geq 0$ とし、 $\rho_m = \sum_n W_{mn} > 0$ 、 $z_m = \sum_n W_{mn} x_n / \rho_m$ とします。

$$\sum_n W_{mn} \|x_n - u_m\|^2 = \rho_m \|z_m - u_m\|^2 + \sum_n W_{mn} \|x_n - z_m\|^2. \quad (5.1)$$

$x_n - u_m = (x_n - z_m) + (z_m - u_m)$ と展開すると、交差項は $\sum_n W_{mn} (x_n - z_m) = 0$ によって消えます。右辺第 2 項は変換 u_m に依存しないため、変換の推定に関して集約は厳密です。近傍代表点への経験的な置換ではありません。[2, §3.3]

5.1 同じ事後を固定した分散更新

M-step 後の移動点を p_m^+ とし、E-step で得た責任度を固定します。分散に関する目的は

$$Q(v) = \frac{E}{2v} + \frac{dN_P}{2} \log v, \quad E = \sum_{m,n} W_{mn} \|x_n - p_m^+\|^2, \quad v = \sigma^2 > 0 \quad (5.2)$$

です。 $\partial Q / \partial v = -E / (2v^2) + dN_P / (2v)$ なので、

$$\sigma_+^2 = \frac{E}{dN_P} = \frac{\sum_m \left(h_m - 2(p_m^+)^T b_m + \rho_m \|p_m^+\|^2 \right)}{dN_P}. \quad (5.3)$$

必ず更新後の座標と実際の次元 d を使います。重心までの距離 $\rho_m \|z_m - p_m^+\|^2$ だけでは、式(5.1)の定数項が欠けて分散を過小評価します。[2, 式 (25)–(27)]

実装では非有限値や大きな負の二乗残差を例外とします。丸め誤差の範囲内の微小な負値は 0 へ戻し、正の分散下限を適用します。この下限は数値的な制約であり、無制約最大尤度解と区別します。点对線・面の剛体目的を使う場合も、分散は空間 Gaussian に対する上式で更新します。

5.2 初期分散

CPD 式の初期分散は

$$\sigma_0^2 = \frac{1}{dMN} \sum_{m,n} \|x_n - p_m\|^2 = \frac{v_X + v_P + \|\mu_X - \mu_P\|^2}{d}, \quad (5.4)$$

ここで $v_X = N^{-1} \sum_n \|x_n - \mu_X\|^2$ 、 $v_P = M^{-1} \sum_m \|p_m - \mu_P\|^2$ です。右辺は同じ全対平均を $O((N+M)d)$ で計算し、大きな共通オフセットによる桁落ちを抑えます。

6 ガウス和をフィルタに変換する入力拡張

一般のガウス和は、観測特徴 f_n 、クエリ特徴 g_m 、観測値 $v_n \in \mathbb{R}^C$ に対して

$$G(g_m) = \sum_n \exp[-\|g_m - f_n\|^2 / 2] v_n \quad (6.1)$$

です。位置のみを使う本版では $f_n = x_n / \sigma$ 、 $g_m = p_m / \sigma$ です。異なる点集合の値を求めるために、FilterReg 式 (13) の拡張を行います。 $G_P = (g_m^T)_m$ 、 $F_X = (f_n^T)_n$ と定義します。[1]

$$F = \begin{bmatrix} G_P \\ F_X \end{bmatrix}, \quad V = \begin{bmatrix} 0 \\ V_X \end{bmatrix}, \quad (V_X)_n = (1, x_n^T, \|x_n\|^2, n_n^T). \quad (6.2)$$

法線を使わないときは最後の d 成分を省きます。値の次元 C は $d+2$ または $2d+2$ です。クエリ側を 0 にするため、拡張集合上のフィルタ結果の先頭 M 行には観測側からの寄与だけが残ります。

1 回の格子構築と多チャンネルの処理で、0 次・1 次・2 次・法線モーメントをまとめて得ます。モーメントごとに異なる近傍探索を行って統計の整合性を失うことは避けます。低水準操作の `start` は、どの行から Splat するかを指定します。登録時は `start = M` とし、クエリ側の値も明示的に 0 とします。

6.1 前回の置換が不適合だった理由

全対評価はこのガウス和を直接計算できますが、permutohedral 格子への埋込み、単体の重心座標、格子上の拡散を実装したことにはなりません。半径探索も同様です。「同じ種類のガウス和を計算する」という共通点だけでは、FilterReg の主要な計算構成を再現したという要件を満たせません。

今回の `direct` は独立した診断用の選択肢として残しますが、FilterReg の既定は本物の格子法です。ACPD の直接 E-step とは目的も選択位置も異なります。

7 Permutohedral 格子：埋込みと単体補間

7.1 次元と埋込み

以下は添付 `probreg` の格子コードと原 FilterReg の補助関数に対応する具体化です。[5, 6] 特徴次元を D とし、 $D+1$ 次元の総和 0 の超平面へ埋め込みます。登録では $D = d$ 、低水準フィルタ単体では $1 \leq D \leq 16$ を許します。

Blur の有無に応じて

$$\gamma_B = (D+1)\sqrt{2/3}, \quad \gamma_0 = (D+1)\sqrt{1/6} \quad (7.1)$$

を使います。入力 $f \in \mathbb{R}^D$ に対して $a_j = \gamma f_j / \sqrt{j(j+1)}$ 、 $j = 1, \dots, D$ と置き、

$$e_0 = \sum_{j=1}^D a_j, \quad e_i = \sum_{j=i+1}^D a_j - i a_i \quad (i = 1, \dots, D) \quad (7.2)$$

とします。 $\sum_{i=0}^D e_i = 0$ です。Blur を省く場合は尺度も変える必要があり、同じ尺度で Blur だけを削除する実装とは異なります。

7.2 最近格子点、順位、重心座標

$h = D + 1$ とします。各 e_i を最も近い h の整数倍 r_i へ丸めます。同距離のときは下側を選びます。残差 $e_i - r_i$ の降順の順位 k_i を求め、 $q = \sum_i r_i / h$ を各順位に加えます。順位が $[0, D]$ から外れた成分について h だけ巻き戻し、対応する r_i も $\pm h$ 補正します。等しい残差は添付スカラー実装の添字順で処理します。

$D + 2$ 成分の作業ベクトル b を 0 にし、各 i について

$$\delta_i = (e_i - r_i)/h, \quad b_{D-k_i} += \delta_i, \quad b_{D-k_i+1} -= \delta_i \quad (7.3)$$

を行い、最後に $b_0 += 1 + b_{D+1}$ とします。先頭 $D + 1$ 成分が単体の重心重み β_j です。

単体の色 $j = 0, \dots, D$ に対応する頂点キーは、独立な最初の D 座標について

$$\kappa_{j,i} = r_i + j - h \mathbf{1}\{k_i > D - j\}, \quad i = 0, \dots, D - 1 \quad (7.4)$$

です。最後の座標は総和 0 から復元可能なので保存しません。特に長さ D のキーから κ_D を読むことは禁止します。

7.3 数値表現と限界

本版は倍精度と 64 ビット整数キーを使い、埋込み座標が 2^{46} を超える場合に整数変換前に拒否します。非有限の重みや許容を超える負の重みも拒否します。添付版の単精度・短整数と比べ、丸め境界付近で異なる単体が選ばれる可能性はあります。全入力に対するビット一致や、ガウス近似の一樣誤差上限を主張しません。

8 Splat・Blur・Slice と原コードの最適化

8.1 標準の三段階

点 i が頂点 κ_{ij} 、重み β_{ij} を持つとき、Splat は

$$L(\kappa) = \sum_{i,j: \kappa_{ij}=\kappa} \beta_{ij} v_i \quad (8.1)$$

です。各点の値をその単体の $D + 1$ 頂点へ線形に分配します。

Blur は軸 $a = 0, \dots, D$ ごとに、存在する格子頂点の値を

$$L^{a+1}(\kappa) = L^a(\kappa) + \frac{1}{2}L^a(\kappa_a^-) + \frac{1}{2}L^a(\kappa_a^+) \quad (8.2)$$

へ更新します。未登録の隣接頂点の値は 0 とします。各軸では別の作業領域へ書き、同じ軸内の走査順に結果を依存させません。独立キーでは全成分を ∓ 1 とした後、 $a < D$ の場合に軸成分だけ $\pm D$ へ置き換えることで隣接キーを得ます。最後の軸は独立キーの範囲外を読みません。

Slice は

$$\tilde{G}_i = \eta_D \sum_{j=0}^D \beta_{ij} L^{D+1}(\kappa_{ij}), \quad \eta_D = \frac{1}{1 + 2^{-D}} \quad (8.3)$$

です。添付 probreg の利得を保持します。逆順の Blur も低水準操作として用意し、原コードと比較します。有限の登録頂点集合で軸演算が常に可換とは仮定しません。

8.2 三つの格子方式を混同しない

方式	構成と由来
permutohedral	拡張点群 $[P; X]$ 、 γ_B 、全軸 Blur、 η_D 。標準の実格子法で、本版 FilterReg の既定です。
permutohedral_noblur	原 FilterReg に対応する観測専用構築です。X のみを γ_0 で Splat し、P は Slice だけ行います。Blur も η_D も適用しません。
probreg	拡張点群の標準格子を調べ、頂点数が $0.015N$ を超えると γ_0 で再構築して Blur を省きます。添付 probreg と同じく η_D は保持します。

原 FilterReg の Blur 省略版も、本物の permutohedral 格子です。半径探索ではありません。観測専用版では X 、値、 σ^2 が固定なら Splat 済み格子を再利用でき、移動点の更新に応じて Slice だけやり直します。分散を更新した場合は特徴尺度が変わるので再構築します。再構築回数は履歴に記録します。

probreg 方式の自動切替は「格子から半径法への切替」ではなく、二種類の格子近似間の切替です。添付原 FilterReg の `updatedvar_withblur.h` は実質的に空であり、そのファイルを完全移植したと称してはいません。標準 Blur の具体的な参照は添付 probreg です。[5, 6]

9 剛体 M-step : twist と局所更新

9.1 点对点目的

事後を固定して、現在の移動点 p_m を剛体増分で更新します。共通の $1/(2\sigma^2)$ を除く目的は

$$E(R_\Delta, t_\Delta) = \sum_m \rho_m \|R_\Delta p_m + t_\Delta - z_m\|^2. \quad (9.1)$$

正規化前後で回転は同じで、並進のみ尺度が変わります。M-step の間に事後を再計算しません。

2D の増分を $\xi = (\omega, v_x, v_y)$ とし、3D では $\xi = (\omega_x, \omega_y, \omega_z, v_x, v_y, v_z)$ とします。左から増分を作用させると、零増分での点のヤコビアンは

$$J_2(p) = \begin{bmatrix} -p_y & 1 & 0 \\ p_x & 0 & 1 \end{bmatrix}, \quad (9.2)$$

$$J_3(p) = \begin{bmatrix} -[p]_\times & I_3 \end{bmatrix}, \quad [p]_\times b = p \times b. \quad (9.3)$$

$\omega \times p = -p \times \omega$ なので回転部分の符号は負です。原論文の skew 説明と式 (16) を混同せず、原コードと変換の有限差分で確認します。[1, §5]

9.2 小さな正規方程式

残差を $r_m = p_m - z_m$ とし、Gauss-Newton の行列と右辺を

$$H = \sum_m \rho_m J_d(p_m)^T J_d(p_m), \quad g = \sum_m \rho_m J_d(p_m)^T r_m \quad (9.4)$$

として、 $H\Delta\xi = -g$ を解きます。 H は 2D で 3×3 、3D で 6×6 です。点数に比例する大きな設計行列を保持する必要はありません。これは原論文式 (18) の大域剛体への特殊化です。

逆行列を明示形成せず、特異値分解で解きます。必要な階数がない場合は、拘束不足として例外にします。独自の剛体正則化や、欠落した回転軸を 0 に固定して成功扱いにする処理は入れません。内側反復数の既定は 1 です。増やした場合も同じ事後を固定して局所線形化を更新します。

9.3 回転の有限更新と合成

2D は $R_\Delta = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix}$ 、3D は Rodrigues 式

$$R_\Delta = I + \frac{\sin \theta}{\theta} [\omega]_\times + \frac{1 - \cos \theta}{\theta^2} [\omega]_\times^2, \quad \theta = \|\omega\| \quad (9.5)$$

を使います。θ が小さいときは係数を級数で評価します。 $t_\Delta = v$ として、

$$R^+ = R_\Delta R, \quad t^+ = R_\Delta t + t_\Delta \quad (9.6)$$

を適用します。これは原コードに対応する回転ベクトルと独立並進の局所更新です。SE(3) の完全な指数写像の並進 $V(\omega)v$ を使う更新とは区別します。一次微分は同じでも有限増分は一般に異なります。

10 点对線・点对面と任意の閉形式解

2D では法線を持つ局所直線、3D では局所平面までの残差を使えます。E-step で平均した法線 $\bar{n}_m$ を固定すると、

$$e_m = \bar{n}_m^T (p_m - z_m), \quad j_m = \bar{n}_m^T J_d(p_m), \quad E_\perp = \sum_m \rho_m e_m^2 \quad (10.1)$$

です。正規方程式は

$$H_\perp = \sum_m \rho_m j_m^T j_m, \quad g_\perp = \sum_m \rho_m j_m^T e_m \quad (10.2)$$

になります。これは FilterReg 式 (9),(10) の 2D/3D 実装です。単一平面など、運動の一部を拘束できない配置では階数不足を検出します。[1]

入力法線は固定点群と同じ行数・次元の単位ベクトルでなければなりません。共通正規化は並進と等方尺度だけなので法線の方法は変えません。法線推定自体は本版の登録処理に含めません。

10.1 任意の Kabsch 解

点对点のみ、solver="kabsch" で閉形式の重み付き剛体解を選べます。これは twist を実装しないための代用品ではなく、別の M-step 解法として明示します。

$$\mu_p = \frac{\sum_m \rho_m p_m}{\sum_m \rho_m}, \quad \mu_z = \frac{\sum_m \rho_m z_m}{\sum_m \rho_m}, \quad (10.3)$$

$$B = \sum_m \rho_m (p_m - \mu_p)(z_m - \mu_z)^T = U \Sigma V^T, \quad (10.4)$$

$$R_\Delta = V \text{Diag}(1, \dots, \det(VU^T))U^T, \quad t_\Delta = \mu_z - R_\Delta \mu_p. \quad (10.5)$$

重心と相互共分散で同じ責任度を用います。 $\sqrt{\rho}$ を行の最小二乗重みに使うことと、重心計算の重みを $\sqrt{\rho}$ に変えることは異なります。添付 probreg の該当箇所との違いを第22節に記録しています。

11 Analytic-CPD の構造化 Taylor 写像

11.1 有限次元の解析関数空間

多重指数 $\alpha = (\alpha_1, \dots, \alpha_d)$ 、 $|\alpha| = \sum_j \alpha_j$ 、 $\alpha! = \prod_j \alpha_j!$ を使い、

$$\phi_q(p; c) = \left[\frac{(p-c)^\alpha}{\alpha!} \right]_{|\alpha| \leq q}, \quad \mathcal{A}(p) = B^T \phi_q(p; c), \quad B \in \mathbb{R}^{S_{d,q} \times d} \quad (11.1)$$

と定義します。基底数と係数数は

$$S_{d,q} = \binom{q+d}{d}, \quad K_{d,q} = d S_{d,q}. \quad (11.2)$$

$q = 10$ で 2D は 66 基底・132 係数、3D は 286 基底・858 係数です。変形係数の数は点数に依存しません。ただし、E-step や当てはめ行列の行数は依然として点数に依存します。[2, §3.4, 付録 A]

本版では正規化後の $c = 0$ を固定し、全総次数を昇順で列挙します。同じ次数では第 1 変数の指数を降順、3D では次に第 2 変数の指数を降順とします。基底評価、推定係数、保存、再適用、微分でこの順序を一致させます。

2D の 2 次基底は

$$\phi_2(u) = \left(1, u_1, u_2, \frac{u_1^2}{2}, u_1 u_2, \frac{u_2^2}{2} \right)^T \quad (11.3)$$

です。3D では 2 次部分が $(u_1^2/2, u_1 u_2, u_1 u_3, u_2^2/2, u_2 u_3, u_3^2/2)$ になります。階乗を基底側に含めるため、係数は微分値型の規約になります。

11.2 剛体成分との関係

1 次までの写像は一般のアフィン写像を含みます。定数項と一次項を削除すると、論文の解析関数空間とは異なる制約付きモデルになります。本版はこれらを削除せず、残差を剛体変位場へ直交投影する制約も既定で加えません。

したがって、剛体段階を固定しても、後続の解析写像には残った並進、回転に似た成分、伸縮、せん断が含まれ得ます。「剛体変数を再推定しない」ことは保証しますが、「有限剛体運動と非剛体運動を一意に分解する」ことは保証しません。この区別が、ユーザー指定の初期姿勢固定と論文の写像空間を両立する仕様です。

12 無正則化の重み付き解析 M-step

現在の点群 $Y_{\text{cur}} = (p_m^T)_m$ 、事後重心 $Z = (z_m^T)_m$ 、責任度 ρ_m から、 $I = \{m : \rho_m > \epsilon_\rho\}$ を作ります。選択行だけの設計行列を Φ 、重み行列を $\Omega = \text{Diag}(\rho_m)_{m \in I}$ とすると、

$$B^* = \arg \min_B \left\| \Omega^{1/2} (\Phi B - Z) \right\|_F^2. \quad (12.1)$$

これは論文式 (20),(22) です。リッジ正則化、最大変位制限、係数射影、更新の減衰は加えません。[2]

$D_w = \Omega^{1/2} \Phi$ 、 $Z_w = \Omega^{1/2} Z$ と置き、 $D_w = U \Sigma V^T$ を特異値分解して

$$B^* = V \Sigma_r^\dagger U^T Z_w \quad (12.2)$$

を計算します。 $\Sigma_\tau^\dagger$ は $s_i > \tau s_{\max}$ の特異値だけを逆数にする数値的擬似逆です。 $\tau = 10^{-12}$ が既定で、実際の階数を履歴に記録します。厳密演算で列フルランクなら唯一の最小二乗解、階数不足なら絶対写像の最小ノルム解です。

正規方程式 $(\Phi^T \Omega \Phi)B = \Phi^T \Omega Z$ は導出上有用ですが、高次数で条件数を二乗するため実装では明示形成しません。SVD は論文 §3.5 と付録 B.7 が認める安定な解法です。全出力座標で同じ分解を共有し、座標ごとに同じ行列を分解し直しません。

12.1 保存する残差係数と、解く目的の区別

恒等写像を表す係数を B_I とすると、 $\Phi B_I = (Y_{\text{cur}})_I$ です。解いた後で $C = B^* - B_I$ を保存し、

$$\mathcal{A}(p) = p + C^T \phi_q(p) \quad (12.3)$$

と再適用します。保存表現が残差であるだけで、解く問題は式(12.1)の絶対写像です。階数不足で最初から最小ノルムの残差係数を解くと、恒等写像に偏った別の解を選ぶことがあるため、その置換はしません。

有効行が少ない場合は、第13節の規則で次数を減らします。最小次数でも行数が足りない場合、上位の反復処理はそれ以前の最良状態を返し、`insufficient_posterior_mass` を記録します。低水準の当てはめ関数へ直接不適合なデータを渡した場合は例外です。

13 次数継続：原コードの剰余割当てまで定義

既定の最大次数 $q_{\max} = 10$ 、反復予算 $T = 55$ 、外れ値重み $w = 0.1$ は論文の標準設定です。低次数の段階を長く、高次数の段階を短く割り当てます。[2, §3.7, §4.1]

一般に最小次数 $q_{\min}$ 、段階数 $J = q_{\max} - q_{\min} + 1$ 、 $U = J(J+1)/2$ とします。 $T = aU + r$ 、 $0 \leq r < U$ に分解し、段階 $j = 0, \dots, J-1$ の初期長さを

$$L_j = a(J-j) \quad (13.1)$$

とします。剰余 r は添付 `DegreeScheduleDecreasingStages` に合わせ、接頭辞長を $J, J-1, \dots, 1$ と下げながら、 $t = \min(\text{接頭辞長}, r)$ として先頭 t 段階を 1 ずつ増やし、 $r \leftarrow r - t$ とします。単純なラウンドロビンや最大剰余法へ置換していません。[7]

$q_{\min} = 1, q_{\max} = 10, T = 55$ では $(L_0, \dots, L_9) = (10, 9, 8, 7, 6, 5, 4, 3, 2, 1)$ です。予算が短い場合は長さ 0 の段階もあり、全ての次数を必ず訪れるとは限りません。

反復 k に対応する予定次数 q_k^{raw} は累積長から決めます。その後、有効行数 $M_s = |I|$ が

$$M_s \geq S_{d, q_k} = \binom{q_k + d}{d} \quad (13.2)$$

を満たすまで次数を下げます。この条件は「未知係数より独立な観測方程式が少ない」ことを避ける必要条件であり、良い条件数や列フルランクを保証しません。幾何学的な階数不足を理由にさらに自動で別のモデルへ切り替えず、SVD の階数を報告します。

同じ予定次数でも、責任度が小さい行の数によって実使用次数は変わり得ます。そのため履歴には `raw_degree` と `degree` の両方を保存します。

14 逐次合成、最良状態、停止判定

14.1 現在の点へ作用する写像列

各 E-step は現在の $Y^{(k)}$ から事後を求め、M-step は同じ現在点上で $\mathcal{A}_k$ を当てはめます。

$$Y^{(k+1)} = \mathcal{A}_k(Y^{(k)}), \quad \mathcal{D}_k = \mathcal{A}_{k-1} \circ \cdots \circ \mathcal{A}_0. \quad (14.1)$$

固定した元座標だけで単一の変位場を再推定する方式ではありません。合成の次数は最大で $\prod_i q_i$ まで増え得ますが、展開して巨大な多項式に変換せず、個々の写像列を保持します。[2, §3.6]

14.2 返却状態の厳密な定義

内部指標を $s_k = \sqrt{d\sigma_k^2}$ とします。初期点群、初期分散、空の写像列を候補として保存し、新しい s_k が真に小さければ最良状態を更新します。返却するのは

$$k_* = \arg \min_{0 \leq k \leq k_{\text{executed}}} s_k \quad (14.2)$$

の点群、分散、 k_* 個の写像列です。同値なら先に得た状態を保持します。途中で悪化した点群だけを巻き戻し、変換係数や分散を最後のままにすることは禁止します。論文 Fig.1 に対応します。[2]

14.3 実装で使う停止条件

相対変化は 10^{-12} を分母の数値補助量として

$$\delta_Y = \frac{\|Y^{(k+1)} - Y^{(k)}\|_F}{\|Y^{(k)}\|_F + 10^{-12}}, \quad (14.3)$$

$$\delta_\sigma = \frac{|\sigma_{k+1}^2 - \sigma_k^2|}{|\sigma_k^2| + 10^{-12}}, \quad \delta_s = \frac{|s_{k+1} - s_k|}{|s_k| + 10^{-12}} \quad (14.4)$$

とします。3 量が許容値未満となる反復が既定で 5 回続けばその次数について収束と判定します。内部指標が許容値未満なら直ちに停止します。その他、最小 6 反復後に、明確な反発、8 回の有意改善なし、反復上限、有効責任度の不足を判定します。

14.4 次数継続と段階終了の区別

次数計画（第13節）は各次数に**最大で**何反復使えるかを与えるものであり、その反復数の消費を要求しません。上の収束判定が立ったとき、計画により高い次数が残っていれば、残りの反復を飛ばして次の次数段階へ進み、待機回数を初期化します。段階全体を終了できるのは、計画上の最高次数で判定が立った場合だけです。この区別を設けないと、計画が低次数へ多くの反復を割り当てると早く「収束」して打ち切られ、予算を増やすほど結果が悪化します。実測では、2 次元・乱数種 90・予算 440 で次数 1 のまま 69 反復目に停止し、アフィン相当の対応付き RMS 0.0374 を返しました。継続を導入した後は同条件で 0.00000 へ収束し、予算 110 から 880 まで結果が単調です。degree_schedule 関数そのものは変更していないため、原コードとの 24 条件比較はそのまま有効です。

14.5 発散した反復を最良状態にしない

解析写像は事後確率が支持する領域でのみ拘束される大域多項式であり、支持を失った点は自由に外挿されます。 σ^2 は責任度重み付きの二次モーメントなので、その点をほとんど見ません。したがって、点群が発散していく途中の状態が s_k を単調に下げ続け、「最良」として採用され得ます。実測では、対応の付かない点群に対して返却点群が正規化座標で 1.8×10^{77} に達しました。そこで、固定点群の最大半径の `divergence\_radius` 倍（既定 100 倍）を超える反復は**採用しません**。段階は `numerical\_divergence` で終了し、それまでに実際に評価された最良状態を返します。これは推定値に対する制約（クリッピング、減衰、罰則）ではなく、既存の待機型停止規則と同種の**停止規則**です。採用された反復の当てはめは、いずれも正則化なしの論文どおりの解のままです。健全な 54 条件では、中間状態を含めて正規化半径 1.84 を超えません。

有意改善は $s_{\text{best}} - s_k > \max(10^{-12}, 10^{-6}|s_{\text{best}}|)$ です。反発は有意改善なしが 5 回以上かつ $s_k > (1 + 10^{-3})s_{\text{best}} + 10^{-12}$ です。有意改善の閾値は待機回数にだけ使い、真の最小指標の保存を省きません。実使用次数が変わったときは待機回数をリセットします。

`history` は試みた全反復、`steps` は返却する最良状態までの写像列です。したがって両者の長さは異なることがあります。反発・改善なしによる終了は「許容値まで収束」と同一視しません。

14.6 外部対応情報は登録に流入させない

点数が同じという理由だけで $x_m \leftrightarrow y_m$ を既知対応と扱いません。原論文付録 B.8 の外部 RMSE 監視は、意味のある既知対応がある評価用の条件です。本ライブラリの順不同登録は内部指標だけで状態を選びます。[2] 第25節の対応付き RMS は評価スクリプトだけが計算し、事後・当てはめ・停止へ渡していません。

15 剛体初期化を固定した非剛体モード

初期姿勢 $T_0(y) = R_0 y + t_0$ が与えられる場合、正規化座標では

$$\tilde{T}_0(u) = R_0 u + \frac{R_0 c_X + t_0 - c_X}{s} \quad (15.1)$$

と共役変換します。FilterReg の増分を合成した最終世界姿勢を $T_F(y) = R_F y + t_F$ として保存します。

解析段階では R_F, t_F を**変更しません**。初期解析点は $u^{(0)} = \mathcal{N}(T_F(y))$ です。最良状態の解析列を $\mathcal{D}_{k*}$ とすると、全変換は

$$T(y) = \mathcal{N}^{-1}(\mathcal{D}_{k*}(\mathcal{N}(T_F(y)))) \quad (15.2)$$

残差変位は $u_F(y) = T(y) - T_F(y)$ 、全変位は $u(y) = T(y) - y$ です。返却する `rigid_transformed` は $T_F(Y)$ 、`transformed` は $T(Y)$ です。

15.1 姿勢と分散を別の初期条件として扱う

ユーザー要求の初期ポーズ継承は、上式の T_F です。分散を引き継ぐこととは別です。解析段階の初期分散は次の優先順位で決めます。

1. 明示的な解析 `sigma2` があれば、それを世界座標の二乗単位として使います。
2. `initialization="auto"`（既定）は、宣言された `method` から決めます。`nonrigid` は `filterreg`、単独 `analytic` は `cpd` へ解決します。実行時の品質判定による切替ではなく、宣言による決定です。

3. `initialization="cpd"` では、剛体更新後の点群に対し CPD 式の全対平均を計算します。ACPD 論文単体のアルゴリズムに対応します。
4. `initialization="filterreg"` では剛体段階の最終分散を継承します。二段階統合に対する明示的な追加選択です。

単独 `analytic` モードには継承元がないため、明示分散なしで `filterreg` 初期化を要求すると入力エラーです。

15.2 既定を段階に整合させる理由

CPD 式の全対初期分散 $\sigma_0^2 = \frac{1}{dNM} \sum_{n,m} \|x_n - y_m\|^2$ は、未整合な点群対に対する推定量です。正規化座標では $\sigma_0^2 \approx 2/d$ 、すなわちカーネル幅が点群の広がりそのものになります。剛体段階が既に収束した後にこれを使うと、事後がほぼ一様になり、各移動点の軟目標が固定点群の重心へ縮退します。実測では、2 次元・乱数種 90 で剛体後の対応付き RMS 0.05664 が 1 反復目に 0.66112 へ跳ね、既定予算では焼き鈍しが戻り切らず 0.17449 で終わりました。姿勢を固定している以上、失った剛体成分を取り戻す保証ありません。

逆に FilterReg 分散の継承は、 σ^2 が 10^{-8} 級まで焼き鈍された値を引き継ぎます。このとき初期内部指標 $s_0 = \sqrt{d\sigma_0^2}$ が達成不能なほど小さくなり、どの反復も最良を更新できず写像列が空で返ります。0.2.0 の 36 条件監査で 13 条件が「剛体結果と同一」だったのはこれが原因です。第14.4節の次数継続を入れると、この設定は逆に強力になります。支持のある少数点（実測では 180 点中 18 点）から当てはめが始まり、 σ^2 が自然に増加して有効点が段階的に広がるためです。実測では 16 反復で全 180 点が有効になり、対応付き RMS は 10^{-6} 級へ達しました。

したがって既定は、単独 ACPD には論文どおりの全対初期化、二段階の残差推定には分散継承、という段階整合の割当としています。どちらの明示指定も引き続き可能で、どちらかが常に有利だとは主張しません。

16 再適用、微分、保存された写像の意味

保存した写像は登録に使わなかった点にも式(15.2)を順番に適用します。元の移動点の個数や点番号へ係数を結び付けていません。

16.1 Taylor 写像のヤコビアン

u の第 j 成分に対する基底微分は

$$\frac{\partial}{\partial u_j} \frac{u^\alpha}{\alpha!} = \begin{cases} u^{\alpha-e_j}/(\alpha-e_j)!, & \alpha_j > 0, \\ 0, & \alpha_j = 0. \end{cases} \quad (16.1)$$

$u_j = 0$ でも評価できる形を使い、基底値を u_j で割りません。残差係数 C_k から

$$J_{A_k}(u) = I_d + C_k^T [\nabla \phi_{q_k}(u)] \quad (16.2)$$

を計算します。 $u_{j+1} = A_j(u_j)$ として連鎖律を使うと、

$$J_T(y) = J_{A_{k_*-1}}(u_{k_*-1}) \cdots J_{A_0}(u_0) R_F. \quad (16.3)$$

正規化の $1/s$ と逆正規化の s は等方なスカラーなので相殺します。実装の返却形は (n, d, d) で、行が出力座標、列が入力座標です。

16.2 保存形式と不変条件

保存形式は版番号 2 の NPZ です。係数列、各次数、 R_F, t_F, c_X, s 、最良分散、段階履歴、終了理由、実装名を含めます。Python オブジェクトの pickle を使わず、読込時は `allow_pickle=False` です。有限性、配列形状、回転の直交性と行列式、履歴の連続性、係数数、最良反復と写像列長を検査します。

世界分散の尺度変換は積の結合順で最下位桁が異なることがあるため、段階分散と結果分散の整合は倍精度機械イプシロンの 8 倍の相対許容で比較します。この検査追加時に完全一致を要求して正常な 12 試験を失敗させた履歴と、修正後の再試験も検証記録に残しています。

16.3 保証しない性質

多項式写像の解析性だけでは、単射性、正のヤコビアン行列式、位相保存、逆写像の存在は保証されません。折り畳みや領域外の大きな外挿が起こり得ます。逆写像を閉形式で提供せず、非有限になる評価は例外とします。ヤコビアンを返すことは、可逆性を検証済みという意味ではありません。

17 入力契約、失敗時の扱いと数値限界

公開 API の順序は `registration(fixed, moving, ...)` です。登録には各点群で少なくとも $d+1$ 点が必要で、有限な 2D/3D 座標を要求します。無限大、NaN、空点群、異なる空間次元、不正な姿勢、範囲外の設定を拒否します。配置による追加の階数不足は実際の M-step で検出します。

`copy=False` では、NumPy の float64、C 連続、適切に整列した配列を要求します。`copy=True` は明示的な型・配置変換を許します。読取専用配列を受け付け、入力を変更しません。ここで `copy=False` はアルゴリズムの作業領域や境界内部での所有コピーを一切作らないという意味ではありません。

C++ 接続は所有した数値領域へ入力をコピーした後に GIL を解放し、出力領域は capsule の所有者が解放する設計です。Rust 接続は NumPy の読取借用から所有行列へコピーし、その後 `Python::detach` 内で数値核を実行します。両方とも GIL を解放した区間で Python 配列の所有権に依存した生ポインタを使わない構成です。0.2.1 以降、両接続はビルドし、driver を介さない境界試験を含めて実行検証しています。[9, 10, 11]

17.1 例外と正常な早期終了

無効入力は `ValueError/TypeError` に対応します。数値的支持不足、負のモーメント、拘束不足、距離や写像のオーバーフローなどは数値例外です。これらを成功値や別方式への切替で隠しません。

ACPD の内部指標の反発や有意改善なしは、最良状態を返す明示的な早期終了です。有限な結果が返ること、`converged=True` であること、外部正解に近いことは別です。剛体段階の収束は、正規化座標での移動 RMS と分散の相対変化が両方許容値以下であることとします。

17.2 厳密 EM の単調性と異なる目的の比較

正確な事後、正確な M-step、同じ目的の更新であれば EM の議論が可能です。本版 FilterReg には格子近似と有限回数の Gauss-Newton が含まれます。ACPD も数値的な階数打ち切り、次数変

更、分散下限、最良状態返却を含みます。実装全体について無条件の尤度単調性や大域最適性は主張しません。

$\sqrt{d\sigma^2}$ はその反復の事後で重み付けした内部量です。対応付き RMS、最近傍距離、点対面残差とは一般に異なります。内部量が改善しても外部正解から遠ざかる可能性は、第25節で実際に確認しています。

18 計算量と実装上の加速

本節の主張はすべて、負荷に依存しない**命令数**（callgrind）で測っています。壁時計時間は比較に使いません。再実行手順と全条件はvalidation/rerun_2026-09-08/SPEED.md にあります。

18.1 M-step の分解：最小ノルム解を保ったまま置換する

第12節の当てはめは、重み付き設計行列 $\Phi_W \in \mathbb{R}^{n_{\text{act}} \times K}$ 、 $K = \binom{q+d}{d}$ に対する正則化なし最小二乗であり、階数不足時には**最小ノルム解**を選ぶという要求が付きます。特異値分解はこれを満たしますが、必要なのは「階数を相対閾値で判定し、最小ノルム解を返す」という性質だけです。完全直交分解

$$\Phi_W = Q \begin{bmatrix} T & 0 \\ 0 & 0 \end{bmatrix} Z^*, \quad T \in \mathbb{R}^{r \times r} \text{ 上三角}, \quad (18.1)$$

も同じ性質を持ち、同じ相対閾値で r を定めます。したがって推定量は変わらず、分解の計算量だけが下がります。実測（実際の重み付き Taylor 設計行列、 $n = 500, d = 3$ ）では階数は全条件で一致し、当てはめ値 $\Phi_W c$ の差は最大 4.6×10^{-12} 、速度は次数 6 で 4.62 倍、次数 8 で 5.47 倍、次数 10 で 9.32 倍でした。監査データの 851 回の当てはめで階数不足は 0 回であり、両者は契約上も実挙動上も同値です。原コードとの M-step 比較はむしろ 5.9952×10^{-15} から 3.9968×10^{-15} へ改善しました。

係数 c 自体は最大 3.3×10^{-3} ずれ得ますが、これは単項式基底の条件数が次数 10 で 2×10^{10} に達するためで、係数が本質的に不定であることの反映です。アルゴリズムが消費するのは当てはめ値であり、保存した写像も同じ基底上で評価されます。

18.2 低ランク近似を採用しない理由

古典 CPD の非剛体 M-step は $M \times M$ のガウス運動核 G の逆行列を要し、 G の固有値が急減衰するため上位 $K \ll M$ 成分による低ランク近似が有効です。[3] 本実装の M-step にその核はなく、対応するのは Taylor 設計行列です。その特異値スペクトルを実測すると、相対閾値 10^{-12} における数値ランクは $d = 2, q = 10$ で 66/66、 $d = 3, q = 8$ で 165/165、 $d = 3, q = 10$ で 286/286 と、常に K と一致します。低ランクではないため、打ち切りは実際に使われている方向を捨てることになり、無条件の加速ではなく精度との交換になります。よって本版では低ランク化を行わず、第18.1節の分解置換で加速しました。

18.3 高速ガウス変換（IFGT）とその適用範囲

第8節の格子は FilterReg 方向の事後にしか使えません。CPD 方向は正規化の向きが逆だからです。そこで CPD 側にも選べる近似として、改良型高速ガウス変換を実装しました。恒等式は

$$e^{-\|q-s\|^2/h^2} = e^{-\|q-c\|^2/h^2} e^{-\|s-c\|^2/h^2} \sum_{\alpha} \frac{2^{|\alpha|}}{\alpha!} \left(\frac{q-c}{h}\right)^{\alpha} \left(\frac{s-c}{h}\right)^{\alpha}, \quad h^2 = 2\sigma^2, \quad (18.2)$$

で、 $|\alpha| \leq p$ で打ち切ります。クラスタリングは一樣格子でセル半対角を要求被覆半径 $r_x = \text{cluster_radius} \cdot h$ に一致させ、構成は $O(n)$ です。被覆半径を h に比例させるため、 σ が小さくなくても打ち切り誤差が破綻しません。実測 ($d=3$ 、既定 $p=5$ 、 $r_x/h=0.25$) の厳密和に対する最大相対誤差は、 σ^2 を 0.5 から 0.0005 まで動かして 5×10^{-6} 以下です。

CPD 方向は 2 回の変換で構成します。まず固定点ごとの支持 $G_j = \sum_i K_{ij}$ を求め、次に $1/(G_j + C)$ で重み付けした $(1, x_j, \|x_j\|^2)$ を移動点へ変換すると、 ρ_i, Px_i, x_i^2 が同時に得られます。

測定に基づく結論。 それでも本ライブラリの想定規模では採用しません。1 ペアあたりの費用は「項数 $\binom{p+d}{d} \times$ 値チャンネル数」で、 $d=3, p=5$ では $56 \times 5 = 280$ 演算です。直接法は 1 ペア約 8 演算なので、IFGT はペア数を 35 分の 1 未満に減らして初めて優位になります。ところが精度は小さい r_x/h を要求し、速度は大きいセル幅を要求するため、両者は正面から対立します。既定値ではセル幅 $0.289h$ 、打ち切り $4h$ で、1 問い合わせあたり方向 14 セル、3 次元で $29^3 \approx 2.4 \times 10^4$ セルとなり、損益分岐は $n \approx 7 \times 10^4$ 点と見積もられます。実測でも FilterReg 方向で直接法の 0.23~0.39 倍、CPD 方向で 0.06~0.09 倍の速度でした。permutohedral 格子が優れるのは、単体分解の費用が d にのみ依存し h に依存しないためで、これが IFGT との本質的な差です。

fgt は明示選択のモードとして提供し、既定にはしません。厳密計算が失敗したときの代替として自動選択することはありません。

18.4 E-step バックエンドの損益分岐

$\sigma^2 = 0.05$ 、命令数、直接法比の倍率（大きいほど速い）：

d	n	permutohedral	観測専用 no-blur	probreg	fgt
2	100	1.31	1.45	1.20	0.36
2	1000	23.00	26.02	16.64	0.39
3	100	1.04	1.37	0.94	0.34
3	1000	11.82	22.37	10.14	0.24

格子と直接法の損益分岐は $n \approx 100 \sim 150$ です。それ未満では直接法が最速であり、既定の permutohedral を小さな点群へ無条件に薦めることはしません。

19 構成、責務分離、両言語の対応

数値核は types、lattice、gaussian、rigid、analytic、registration に分離します。格子は値チャンネルの意味を知らず、ガウス統計層がモーメントを組み立て、剛体・解析の当てはめ層がそれぞれの制約を解きます。統合層だけが二段階の順序と姿勢固定を管理します。

C++ では Eigen、Rust では nalgebra を数値線形代数に用います。Rust 数値核には Python 依存も C++ 呼出しもなく、unsafe を禁止しています。PyO3 と nanobind の接続部分は引数検査、所有コピー、GIL の管理、結果の型変換に限定します。

共通 Python 層は不変の設定型、配列契約、結果の評価・保存を担当します。engine="cpp" と engine="rust" は明示選択です。指定した拡張がない場合、他方や Python 参照計算へ黙って切り替えません。試験用の C++ 実行形式への接続は tests にのみあり、製品の実行経路ではありません。

19.1 SOLID・KISS・DRY の適用範囲

単一責務は上記の層分離で実現します。低水準のガウス計算や写像評価を登録処理から独立して再利用できます。点对点／点对面は同じ twist 組立てを共有し、Taylor の全出力座標は一度の分解を共有します。2D/3D は共通の式を使い、運動学に必要な小さい差だけ分岐します。

ただし、独立した二言語実装という要求のため、数式の実装が言語間で重複すること自体は避けられません。共通の原コード由来数値データと操作仕様で整合を保ちます。不要な継承階層や汎用最適化枠組みを加えず、少数の型と関数で構成します。これらは設計上の選択であり、全ての拡張要求に対する完全な SOLID 準拠を証明するものではありません。

20 操作仕様と環境の再現

20.1 主要な既定値

設定	既定	意味
剛体最大反復	60	FilterReg 外側反復の上限です。
剛体解法・距離	twist・点对点	2D/3D の局所剛体更新です。
剛体内側反復	1	同じ事後を固定した Gauss-Newton 回数です。
剛体許容値・ w	10^{-7} , 0.1	正規化座標の停止判定、外れ値重みです。
剛体分散下限	10^{-8}	正規化座標の二乗単位です。
解析最大反復・次数	220・1 から 10	三角形の段階長を使います。次数継続により上限として働きます。
解析許容値・ w	10^{-7} , 0.1	内部指標・相対変化、外れ値重みです。
解析分散初期化	auto	nonrigid は FilterReg 継承、単独 analytic は CPD 式。明示指定も可。
解析 E-step	direct	厳密な直接事後。明示選択で fgt 近似も可。
発散半径	100	固定点群半径の何倍で反復を拒否するか。停止規則です。
解析分散下限	10^{-12}	正規化座標の二乗単位です。
階数閾値・有効質量	各 10^{-12}	完全直交分解の相対閾値、責任度の行選択です。
最小反復・安定待機	6・5	微小変化による停止判定です。
改善なし待機	8	有意改善が得られない反復数です。

全パラメータと単位はdocs/API.md、実行例はexamples/register_points.py、C++ 単独例はexamples/native_core.cpp、Rust 単独例はrust/crates/acpd-core/examples/register_points.rs にも記載します。

20.2 pixi 環境

cpp 環境は C++ コンパイラ、CMake、ninja、nanobind を含み、Rust を要求しません。rust 環境は Rust、maturin、PyO3 側の依存を使い、C++ 数値核や nanobind を要求しません。default は統合環境、docs は文書専用環境です。環境・機能・タスクの分離は pixi の構成規約に従います。[12]

想定する操作は次のとおりです。これらの pixi 経由の依存解決は今回の環境では実行できていません。

```
pixi run -e cpp test-cpp
pixi run -e rust test-rust
pixi run test
pixi run -e docs docs
```

pixi.lock と Cargo.lock は未生成です。バージョン範囲と一部の固定バージョンを指定したソース設定を、ロック済み環境と呼びません。C++ の実際の検証は、利用可能だった GNU C++ 14.2、CMake、Python 3.13.5 で行いました。pixi 設定の Python 3.12 環境そのものの検証とは区別します。

20.3 拡張なしで実行済みの数値核検証

```
cmake -S cpp -B build/core -DACPD_BUILD_PYTHON=OFF \
  -DACPD_BUILD_TESTS=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build/core --parallel 2
ctest --test-dir build/core --output-on-failure
```

さらに試験用実行形式を明示して Python 試験を実行します。この経路を nanobind の検証と呼ぶことは禁止します。C++ 静的ライブラリのインストールと、別の CMake 利用側からの find_package(acpd) および 2D/3D の再適用例も確認します。

21 計算量と性能の主張の範囲

ACPD の直接 E-step は $O(MNd)$ 時間です。全事後行列を保存せず、1 観測点分の重みと ρ, b, h を集計するため、点群を除く主要な作業領域は $O(Md)$ です。

解析当てはめで $K = S_{d,q}$ 、有効行数を M_s とすると、基底・設計行列は $O(MK + M_s K)$ 領域を使い、密な最小二乗は概ね $O(M_s K^2 + K^3)$ です。点数に依存しないのは K の側であり、全登録処理が点数に依存しないわけではありません。[2, 付録 A]

格子では拡張点数を $L_P = N + M$ 、頂点数を L_V 、値チャンネル数を C とすると、単体の順位計算が $O(L_P D^2)$ 、Splat と Slice が $O(L_P (D + 1) C)$ 、Blur が $O(L_V (D + 1) C)$ です。ハッシュ探索は平均的な定数時間を仮定します。最悪の場合のハッシュ衝突まで含む厳密線形時間保証ではありません。

観測専用 Blur 省略版は初回の観測側構築後、分散固定なら各反復でクエリの単体計算と Slice を行います。proberg 方式は切替時に二つの格子を構築することがあるので、その回数を 1 と偽って報告しません。

twist の未知数 p は 2D で 3、3D で 6 です。点对点の正規方程式組立ては $O(Mdp^2)$ 、小行列の分解は $O(p^3)$ です。解析写像列 k_* 個の再適用は $O(Md \sum_{j < k_*} S_{d,q_j})$ です。

本書の比較誤差と合成例の時間から、FilterReg 論文の「ICP より何倍速い」という性能を推定しません。試験用実行形式の往復にはプロセス起動と直列化が含まれ、論文の実行時間と比較できません。GPU や多スレッド性能も測定していません。

22 原資料との差分と選択の根拠

対象	添付資料の内容・不整合	本版の選択
FilterReg 記号	原論文は X が移動側、 Y が固定側です。	公開操作では固定 X 、移動 Y に統一し、式を正しく置換します。
ガウスの係数	PDF 使用時と非正規化 Gaussian 和で外れ値定数が異なります。	$(2\pi\sigma^2)^{d/2}$ を定数側に含めます。
原 FilterReg 省略版	観測のみ構築する実格子・Blur 省略です。	独立した <code>permutohedral_noblur</code> として実装し、標準版と区別します。
probreg 切替	拡張入力・頂点数による切替と利得を使います。	probreg 選択でこの規則を保持します。
probreg スカラー	独立キーの範囲外読出し、内部への <code>start</code> 未転送があります。	範囲外読出しを避け、開始位置を正しく適用します。参照試験ではクエリ値を 0 にします。
格子の精度	単精度と短整数キーです。	倍精度、64 ビットキー、変換前検査を使います。ビット一致とは呼びません。
原 FilterReg 外れ値	固定定数 0.2 と $m_0 < 0.01$ の処理があります。	論文の w, N, M, σ による責任度を用い、固定設定の全出力一致とは称しません。
probreg 分散	更新前の点と固定値 3 を使う箇所があります。	更新後の点と d で式(5.3)を評価します。
probreg Kabsch	重心と共分散で重みの次数が一致しません。	任意 Kabsch では一貫した責任度を使用します。既定は <code>twist</code> です。
ACPD 解法	添付 2D は正規方程式 LDLT、3D はブロック QR です。	論文で許される共有設計行列の SVD を使います。正則化は加えません。
ACPD 最良状態	コードは有意改善条件と保存が結び付きます。	論文 Fig.1 の真の最小指標を保存し、有意改善は待機回数だけに使用します。
ACPD 外部 RMSE 統合処理	同じ点数で対応付き RMSE を自動使用する箇所があります。 二つの論文単体には本ユーザー指定の二段階処理はありません。	論文の既知対応という前提を優先し、順不同登録には使用しません。 姿勢固定を統合仕様として追加し、分散継承は任意選択とします。

これらはdocs/SOURCE_DIFFERENCES.mdにも列挙します。論文と原コードの両方が存在する場合に、都合のよい方だけを選んで同一実装と称することを避けるためです。

23 厳格レビューと是正内容

前回版の受入判定は**不合格**です。理由は努力量ではなく、要求された方法を識別する主要構成が欠落していたことです。注意書きを付ければ主要アルゴリズムの削除が許される、という扱いは適切ではありませんでした。

ID・重大度	前回の欠陥	是正
R01・P0	permutohedral がなく、全対評価／半径探索のみでした。	単体・重心座標・Splat・全軸 Blur・Slice と観測専用省略版を両言語で実装しました。
R02・P0	剛体更新が SVD だけで twist がありませんでした。	SE(2), SE(3) の twist と点对線・面を実装しました。
R03・P0	固定参照、正則化、剛体直交制約、変位上限などが納品間で変化していました。	無正則化の絶対写像当てはめと現在点への逐次合成に統一しました。
R04・P1	最大次数 8、既定 3 で論文の次数 10 を指定できませんでした。	2D66 基底、3D286 基底まで実装しました。
R05・P1	次数割当の剰余規則が原コードと異なりました。	三角形の接頭辞割当まで照合しました。
R06・P1	最終状態を返すだけで最良状態の復元がありませんでした。	点群・分散・写像列を一緒に復元します。
R07・P1	姿勢継承と分散継承を同一の必須条件としていました。	両者を分離し、論文式初期分散と任意の継承を区別しました。
R08・P1	自己試験だけで原実装との成分照合がありませんでした。	原コードから独立した比較用実行形式を作成しました。
R09・P1	Python 接続と pixi が未実行でした。	0.2.1 で実 nanobind / PyO3 拡張と pixi 4 環境を実行し解消しました。0.2.0 の阻害記録は保持しています。
R10・P2	三系統の名前・設定・保存仕様が混在しました。	v0.2.0 の正本、移行説明、入力指紋、追跡表に統一しました。

レビューと作業書は実装前に作成し、WORK_ORDER.sha256 で固定しました。判定が難しい項目を後から削除したり、環境がないことを理由に完了条件の意味を変えたりしません。

24 原実装比較と実行済み試験

24.1 参照計算の独立性

添付 probreg の permutohedral.cpp/.h をそのまま使う比較用実行形式、原 FilterReg の Blur 省略単体計算を使う実行形式、Analytic-CPD の Algo.h/Fitting.h から対象関数を抽出した実行形式を作成しました。製品の新しい数値核をリンクしていません。

Analytic-CPD の文字コードを GB18030 から UTF-8 へ変換し、関数本体を保持したまま比較用名前空間へ入れ、Windows 固有の __declspec を空定義にします。FilterReg の補助関数は無関係な CUDA ヘッダを小さいキー宣言で置き換えます。probreg はスカラー経路を明示します。抽出元、関数名、元の行範囲、元ファイルと抽出部分の SHA256、ビルドコマンドを記録します。

元の実行環境全体、描画処理、データ読込、元の停止判定を丸ごと再現する比較ではありません。以下は原コードの**成分計算**に対する照合です。

24.2 比較結果

対象	条件数	最大絶対差
permutohedral の出力	32	9.2059×10^{-7}
原 FilterReg の単体重み	2	2.4017×10^{-7}
CPD 事後統計	6	3.5527×10^{-15}
Taylor 基底	12	5.5511×10^{-17}
解析 M-step の更新点	8	5.9952×10^{-15}
次数割当	24	0 (全条件で一致)

全 84 条件で、比較スクリプトが定義した数値許容と構造検査を満たしました。格子頂点数、単体キー、次数列も対応する条件で一致しました。参照格子は単精度、本版は倍精度です。約 9.21×10^{-7} という値は、厳密なガウス和に対する近似誤差ではありません。同じ種類の格子近似を行う原コードと今回の移植の差です。

固定比較データの SHA256 は

```
a2873d3a1527c11298c0238a62a0268b25fb946e9b0de281621e3ea1baf4a187
```

です。数値・許容・各ケースは `tests/fixtures/upstream.json` と `validation/upstream_comparison.json` に保持します。

24.3 検査単位と重複計上の防止

C++ 単体は **100 項目の検査**で、CTest 上では 1 つの実行形式です。Release でも検査を無効化しない仕組みです。アドレス検査・未定義動作検査・リーク検査もこの数値核で実行しています。

Python 経由の **193 試験**を C++ エンジンで実行します。格子比較、事後、基底、当てはめ、twist の有限差分、次数低減、姿勢固定、保存復元、入力例外などを含みます。84 条件はこれらの試験と重なるため、独立な成功例として足し合わせていません。

Rust には 84 参照条件と性質検査、計 90 個の試験関数を収録し、`cargo test` で実行しています。実 nanobind / PyO3 の境界 13 条件は driver を使わない経路で、両実装の一致は 5 バックエンド × 2 次元の 10 条件で実行しました。Rust エンジン経由の Python 試験は 153 件です。欠けた拡張を自動スキップして試験全体を成功表示しない設定にしています。

25 合成データでの精度監査：悪化例も保持

原コードとの成分一致とは別に、二段階処理が実際に精度を改善するかを確認しました。2D/3D それぞれ 3 乱数種、点数 180、3 種類の計算（標準格子、観測専用 Blur 省略、直接法）、3 種類の解析初期分散（auto、CPD 式、FilterReg 継承）で、計 54 条件です。proberg 自動切替はこの精度監査には含めず、成分試験で扱っています。

既知変形は第 1 座標に $0.03y_2^2 + 0.06$ を加えるものです。剛体の初期分散は世界単位で 0.08、解析の最大反復は既定、最大次数 10 です。全設定、点群生成、対応付き RMS と対称最近傍 RMS、時間、終了理由を `tools/benchmark_synthetic.py` と JSON に保存します。

剛体段階後との比較	0.2.0 (36 条件)	現行 (54 条件)
対応付き RMS が 10^{-8} を超えて改善	17	54
対応付き RMS が 10^{-8} を超えて悪化	6	0
差が 10^{-8} 以内	13	0
数値例外	0	0
最終対応付き RMS の中央値	0.0408	1.02×10^{-7}
同・最悪値	0.348	1.63×10^{-6}

25.1 0.2.0 で悪化していた条件の原因

標準格子・2D・乱数種 90・CPD 式初期分散では、対応付き RMS は初期 0.10933、剛体後 0.05664、解析後 0.17449 で、**剛体段階より悪化**していました。原因は二つで、いずれも近似や正則化の不足ではなく、第14.4節の次数継続と第15.2節の段階整合の欠如でした。是正後、同条件は 10^{-6} 級へ収束し、監査 54 条件のすべてが剛体段階を改善します。悪化例を削除して成功率を上げたのではなく、原因を特定して除いた結果です。是正前の値と根拠は `validation/as_delivered_v0.2.0/` に保存しています。

一方、剛体段階より必ず良くなることを一般に保証するものではありません。第14.5節のとおり、対応の付かない点群対では発散が起り得ます。その場合は `numerical\_divergence` で停止し、評価済みの最良状態を返します。

25.2 この監査から言えること・言えないこと

内部の $\sqrt{d\sigma^2}$ を最小にすることは、既知対応の RMS を最小にすることと同じではありません。局所的な確率最適化、広い初期分散、柔軟な解析写像は、対応の曖昧な形状で縮退した解を選び得ます。逆に、小さい分散継承では有用な残差更新が進まない場合があります。

この監査は原論文の形状・欠損・外れ値・大変形実験の再現ではなく、一般的な成功率の推定でもありません。無正則化という忠実性を維持したまま、実際に起こった限界を報告するための小規模な検査です。新しい正則化や対応制約が必要な応用では、それを別の拡張仕様として評価する必要があります、本版へ黙って加えて成功例に変えることはしません。

26 完了条件の判定と残る障害

DoD は `docs/review/WORK_ORDER.md` で実装前に固定しました。PASS は要求した証拠を得た項目、SOURCE_ONLY はコードと静的確認だけ、BLOCKED は環境障害による未達です。SOURCE_ONLY や BLOCKED を実行済み PASS へ読み替えません。

ID	判定	内容
D01	PASS	厳格レビュー、固定作業書、入力指紋、対応表を作成しました。
D02	PASS	両言語に実格子を実装し、C++ は原コード 32 条件を再現、Rust も同じ固定オラクルに合格しました。
D03	PASS	逆事後、分散、twist、点对線・面を実装し、有限差分と式照合を両言語で実行しました。

ID	判定	内容
D04	PASS	無正則化の解析当てはめ、次数計画、逐次合成、最良状態を実装・照合しました。
D05	PASS	姿勢固定、写像再適用、保存復元、ヤコビアンを実 nanobind / 実 PyO3 の両方で確認しました。
D06	PASS	C++ ビルド、原コード成分比較、異常系、メモリ検査を実行しました。
D07	PASS	実 nanobind 拡張をビルド・インポートし、driver を使わない境界 13 試験を含めて合格しました。
D08	PASS	Rust 数値核 90 試験と実 PyO3 経由 153 試験、実装間比較 10 条件を実行しました。先行指示の実行免除は使用していません。
D09	PASS	pixi の default / cpp / rust / docs 4 環境を解決し、pixi.lock と rust/Cargo.lock を生成しました。
D10	PASS	本 LaTeX/PDF、参考資料、差分、検証と限界を記述し、再コンパイルと描画検査の結果を検証記録に保存しています。
D11	PASS	クリーン展開から両言語をビルドし、試験一式を再実行しました。最終版のコード指紋一致と配布内容も検査しています。

総合判定は PASS です。 0.2.0 で BLOCKED だった D07・D09 と、実行免除だった D08 は、ビルドホストに nanobind・pixi・cargo が揃った環境で実際に実行して解消しました。完了条件を弱めていません。

26.1 0.2.0 時点の阻害と、その記録の保持

0.2.0 では、実 nanobind 構成の CMake が No module named nanobind で失敗し、指定バージョンの取得も失敗しました。pixi install -e cpp は実行ファイルがなく開始できず、Rust コンパイラと Cargo も存在しませんでした。いずれもビルドホスト側の制約であり、ソースの欠陥ではありませんでした。当時の判定、コマンド、戻り値、ログはvalidation/as_delivered_v0.2.0/ にそのまま保持し、現在の合格記録で消していません。現行の実行記録はvalidation/rerun_2026-09-08/にあります。

26.2 DoD 達成後に行った是正と高速化

DoD の固定条件を変えないまま、第14.4節の次数継続、第15.2節の段階整合初期化、第14.5節の発散停止を導入し、精度監査は 54 条件すべてで剛体段階を改善するようになりました。速度は決定的な命令数で計測し、格子のアロケーション除去、M-step の完全直交分解化、直接法 E-step の改善により解析段で最大 3.20 倍としました。経緯と数値はCHANGELOG.md とvalidation/rerun_2026-09-08/SPEED.md にあります。

27 資料識別、ライセンス、再照合の手順

アルゴリズムの根拠は添付版です。主要な指紋を以下に示し、完全な SHA256 とファイル容量はdocs/SOURCE_MANIFEST.json に記録します。

資料	SHA256 先頭 16 桁
FilterReg 論文 (2019 年 7 月 16 日 v3)	8f88e6854f484f01e
Structured Analytic CPD 論文 (2026 年 5 月 15 日 v2)	1c71045fa1aa27020
FilterReg-master.zip	5575eb0fc2a7aa14d
probreg-master(1).zip	86ed33276127ba10e
Analytic-CPD-main(1).zip	e5508e6106739ec78
cpd3d_rs-main.zip	5f63b4e7a6c269331

新規統合部分は MIT として配布し、permutohedral 由来部分の BSD-3-Clause、Analytic-CPD の MIT、同梱 Eigen の各通知を保持します。プロジェクト全体の MIT 表示で第三者の条件を上書きしません。原 FilterReg のプロジェクト全体を新しいライセンスで再配布することは避け、比較用の対象関数はユーザーが持つ添付 ZIP から再抽出する道具を用意します。原 PDF、大きな原データ、フォントファイル、作業中の実行形式はソース ZIP へ含めません。

原コードの再照合は、元添付を置いたディレクトリを指定してtools/build_reference.py を実行し、tools/generate_fixtures.py で比較値を再生成します。tools/validate_sources.py が製品側数値核との差を検査します。Rust 用の同じ比較値はtools/generate_rust_fixtures.py で生成します。原コードの必要な変更と抽出範囲はvalidation/oracle_provenance.json に保持します。

0.2.0 の SOURCE_ONLY / BLOCKED 記録は消さず、新しい実行条件・ログ・指紋を伴う判定として追加しています。検証履歴はそれによって保たれます。

参考資料

- [1] W. Gao and R. Tedrake, *FilterReg: Robust and Efficient Probabilistic Point-Set Registration using Gaussian Filter and Twist Parameterization*, CVPR 2019. 本書の基準は添付 arXiv:1811.10136v3、2019 年 7 月 16 日、10 ページです。特に §3–5、式 (6)–(19) を参照しています。
- [2] W. Feng and H. Zheng, *Structured Analytic Coherent Point Drift for Non-Rigid Point Set Registration*, 添付 arXiv:2605.00934v2、2026 年 5 月 15 日、22 ページです。§3、Fig.1、付録 A–C を主要な基準としています。未添付の更新版へ置き換えていません。
- [3] X.-W. Feng, D.-Z. Feng and Y. Zhu, *Fast Coherent Point Drift*, arXiv:2006.06281, 2020 年 6 月。古典 CPD の $M \times M$ ガウス運動核に対する対応制約と低ランク近似 (式 (19)–(20)) を参照しました。本書第18.2節のとおり、その低ランク化は本実装の Taylor 設計行列には移りません。
- [4] V. I. Morariu, B. V. Srinivasan, V. C. Raykar, R. Duraiswami and L. S. Davis, *Automatic online tuning for fast Gaussian summation*, NIPS 2008。第18.3節の展開と打ち切りの一般的な形はこの系統の改良型高速ガウス変換に基づきます。クラスタリング方式と既定値は本実装の選択であり、原著の自動調整をそのまま移植したものではありません。
- [5] ユーザー添付 probreg-master(1).zip。probreg/filterreg.py、probreg/cc/kabsch.cc、third_party/permutohedral/permutohedral.cpp および permutohedral.h を照合しました。permutohedral 部分の著作権・BSD-3-Clause 通知を保持しています。
- [6] ユーザー添付 FilterReg-master.zip。geometry_utils/permutohedral_common.hpp、corr_search/gmm/、kinematic/rigid/rigid_point2point_cpu.cpp、kinematic/rigid/rigid_point2plane_cpu.cpp 等を照合しました。本書の「原 FilterReg 省略版」は観測専用格子を意味します。
- [7] ユーザー添付 Analytic-CPD-main(1).zip。Analytic_CPD/Algo.h、Analytic_CPD/Fitting.h。Wei Feng、2026 年、MIT 通知を保持しています。元の文字コードと抽出行範囲は比較記録に記載しています。
- [8] ユーザー添付 cpd3d_rs-main.zip。Python からの Rust 数値核利用の構成を参照しました。本版は完全な置換互換ではなく、元の IFGT、類似変換の尺度、旧保存形式をそのまま再現するものではありません。
- [9] nanobind 公式文書、*The nb::ndarray class*、所有権および GIL の API 文書。<https://nanobind.readthedocs.io/en/latest/ndarray.html>。依存 API の確認のみに使用し、登録アルゴリズムの根拠には使用していません。本版の指定バージョンは 2.12.0 です。
- [10] PyO3 公式 API 文書、Python への接続、所有オブジェクト、Python::detach。<https://docs.rs/pyo3/0.28.3/pyo3/>。依存 API の確認資料です。実際のコンパイル成功の証拠ではありません。
- [11] rust-numpy 0.28.0 公式 API 文書、*PyUntypedArrayMethods*、*PyReadonlyArray*。<https://docs.rs/numpy/0.28.0/numpy/trait.PyUntypedArrayMethods.html>。整列、C 連続、配列形状と読取借用を確認しています。
- [12] Pixi 公式文書、*Pixi manifest*、環境・機能・タスク。https://pixi.prefix.dev/latest/reference/pixi_manifest/。設定仕様の確認資料です。依存解決済み環境やロックファイルの代替ではありません。